

MÓDULO 3: Structs, Enums y Pattern Matching

Aquí es donde Rust empieza a brillar como lenguaje para **modelar dominios de forma expresiva**. Los enums de Rust no son como los "enums" de C/Java (simples etiquetas numéricas) — son **tipos algebraicos de datos** (ADTs), tan potentes como los de Haskell o OCaml. Esto es lo que hace que `Option<T>` y `Result<T, E>` sean tan poderosos.

3.1 Structs: Agrupando Datos Relacionados

♦ Struct clásica (con campos nombrados)

```
struct Usuario {
    nombre: String,
    email: String,
    edad: u32,
    activo: bool,
}

fn main() {
    let usuario1 = Usuario {
        nombre: String::from("Ana"),
        email: String::from("ana@rust.dev"),
        edad: 28,
        activo: true,
    };
    println!("{}", usuario1.nombre, usuario1.edad);

    // Para mutar CUALQUIER campo, toda la instancia debe ser mut
    let mut usuario2 = Usuario {
        nombre: String::from("Luis"),
        email: String::from("luis@rust.dev"),
        edad: 30,
        activo: false,
    };
    usuario2.activo = true;

    // Struct update syntax: crear una instancia a partir de otra
    let usuario3 = Usuario {
        nombre: String::from("Carla"),
        ..usuario2 // toma el resto de los campos de usuario2 (¡mueve
email, que no es Copy!)
    };
    println!("{}", usuario3.email);
    // println!("{}", usuario2.email); // ✗ email fue MOVIDO a usuario3
    println!("{}", usuario2.activo); // ✓ bool es Copy, usuario2 sigue
teniendo edad/activo válidos
}
```

Punto sutil importante: la sintaxis `..usuario2` mueve los campos que no son Copy (como `String`). Esto es consistente con todo lo del Módulo 2 — no hay reglas especiales para structs, el ownership funciona campo por campo.

♦ Struct tupla (sin nombres de campo)

```
struct Punto2D(f64, f64);
struct Color(u8, u8, u8);

fn main() {
    let origen = Punto2D(0.0, 0.0);
    let rojo = Color(255, 0, 0);
    println!("x={}, y={}", origen.0, origen.1);
    println!("R={}", rojo.0);
}
```

Útil cuando el "nombre del tipo" ya lo dice todo y nombrar cada campo sería redundante.

♦ Struct unitaria (sin campos)

```
struct Marcador; // no ocupa memoria, útil para traits (lo verás en
Módulo 5)
```

```
fn main() {
    let _m = Marcador;
}
```

♦ Métodos con impl

```
struct Rectangulo {
    ancho: f64,
    alto: f64,
}

impl Rectangulo {
    // "Método asociado" (constructor por convención, no es especial
    en el lenguaje)
    fn nuevo(ancho: f64, alto: f64) -> Self {
        Self { ancho, alto } // field init shorthand: ancho: ancho
    -> ancho
    }

    // Método que toma &self: solo LEE la instancia
    fn area(&self) -> f64 {
        self.ancho * self.alto
    }

    // Método que toma &mut self: MODIFICA la instancia
    fn escalar(&mut self, factor: f64) {
        self.ancho *= factor;
        self.alto *= factor;
    }
}
```

```
fn main() {
    let mut rect = Rectangulo::nuevo(10.0, 5.0);
    println!("Área: {}", rect.area());
    rect.escalar(2.0);
    println!("Área tras escalar: {}", rect.area());
}
```

Diagrama mental: self, &self, &mut self

```
fn metodo(self)          -> TOMA ownership de la instancia (la consume,
ya no se puede usar después)
fn metodo(&self)          -> PRESTA la instancia inmutablemente (solo
lectura)
fn metodo(&mut self)      -> PRESTA la instancia mutablemente (puede
modificarla)
```

&self es, por lejos, el más común. self (sin &) se usa para builders que "consumen" el objeto paso a paso, o conversiones (into_algo(self)).

Error típico del compilador #1: Olvidar mut en la instancia

```
fn main() {
    let rect = Rectangulo::nuevo(10.0, 5.0);
    rect.escalar(2.0); // ✗
}
```

```
error[E0596]: cannot borrow `rect` as mutable, as it is not declared
as mutable
```

```
--> src/main.rs:3:5
  |
2 |     let rect = Rectangulo::nuevo(10.0, 5.0);
  |         ---- help: consider changing this to be mutable: `mut
rect`
3 |     rect.escalar(2.0);
  |     ^^^^^^^^^^^^^^^^^ cannot borrow as mutable
```

Mismo principio del Módulo 2: mutabilidad explícita, ahora aplicada a instancias completas.

3.1.b Patrón Newtype: Type Safety de Dominio y Encapsulación

El patrón **Newtype** consiste en envolver un tipo primitivo existente dentro de una *struct tupla* de un solo campo. Proporciona dos ventajas críticas en código de producción:

1. **Type Safety en tiempo de compilación:** Evita el antipatrón de *primitive obsession* (por ejemplo, pasar accidentalmente un `id_usuario` en lugar de un `id_cuenta`, aunque ambos sean `u64`).
2. **Base de la Regla del Huérfano (Orphan Rule):** Permite implementar traits externos sobre tipos de la librería estándar o crates de terceros (se profundiza en el Módulo 5).
3. **Cero costo en runtime:** El compilador optimiza la estructura y la sustituye directamente por el tipo primitivo subyacente.

Diagrama Mental: Type Safety con Newtypes

SIN NEWTYPE (Primitive Obsession):
fn transferir(origen: u64, destino: u64, monto: f64)
Llamada: transferir(destino_id, origen_id, 100.0); //  Compila,
pero intercambió cuentas en silencio.

CON NEWTYPE:
struct CuentaOrigenId(u64);
struct CuentaDestinoId(u64);
fn transferir(origen: CuentaOrigenId, destino: CuentaDestinoId,
monto: f64)
Llamada: transferir(destino_id, origen_id, 100.0); //  ERROR DE
COMPILACIÓN: Tipos incompatibles.

Código Compilable

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct UsuarioId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct FacturaId(pub u64);

struct Factura {
    id: FacturaId,
    cliente: UsuarioId,
    total: f64,
}

fn emitir_recibo(factura: FacturaId, usuario: UsuarioId) {
    println!("Recibo emitido para Factura #{} a nombre de Usuario  
#{}", factura.0, usuario.0);
}

fn main() {
    let user_id = UsuarioId(101);
    let factura_id = FacturaId(9001);

    // Llamada correcta
    emitir_recibo(factura_id, user_id);

    // Intentar intercambiar los IDs provocará que el compilador
    rechace el build:
    // emitir_recibo(user_id, factura_id);
}
```

⚠ Error típico del compilador: Intercambio de tipos newtype equivalentes

```
fn main() {
    let user_id = UsuarioId(101);
    let factura_id = FacturaId(9001);
    emitir_recibo(user_id, factura_id); // ✗
}
```

```
error[E0308]: mismatched types
--> src/main.rs:20:19
   |
20 |     emitir_recibo(user_id, factura_id);
   |     ^^^^^^^^^^^^^^^^^^^^^ expected `FacturaId`, found
   |     `UsuarioId`
   |
   = arguments to this function are incorrect
```

3.1.c Derivación de Default e Inicialización con `..Default::default()`

El trait `std::default::Default` define un valor base o estado inicial predecible para una estructura. La macro `#[derive(Default)]` implementa el trait automáticamente si todos los campos de la estructura también implementan `Default`.

Se combina con la **Struct Update Syntax** (`..`) para instanciar estructuras con valores por defecto y sobrescribir únicamente los campos de interés.

Código Compilable

```
#[derive(Debug, PartialEq)]
enum NivelLog {
    Info,
    Debug,
    Error,
}

impl Default for NivelLog {
    fn default() -> Self {
        NivelLog::Info
    }
}

#[derive(Debug, Default)]
struct ConfiguracionServidor {
    host: String,           // Default: ""
    puerto: u16,            // Default: 0
    max_conexiones: u32,    // Default: 0
    tls_habilitado: bool,   // Default: false
    nivel_log: NivelLog,    // Default: NivelLog::Info
}
```

```
fn main() {
    // 1. Instancia estándar con valores por defecto
    let config_default = ConfiguracionServidor::default();
    println!("Default completo: {:?}", config_default);

    // 2. Instanciación explícita con sobreescritura selectiva
    let config_produccion = ConfiguracionServidor {
        host: String::from("0.0.0.0"),
        puerto: 8080,
        tls_habilitado: true,
        ..Default::default() // max_conexiones y nivel_log toman sus
valores default
    };
    println!("Producción: {:#?}", config_produccion);
}
```

3.2 Enums: Modelando Variantes

A diferencia de C, los enums de Rust pueden **llevar datos asociados**, y cada variante puede tener un tipo de dato distinto:

```
enum Forma {
    Circulo(f64), // radio
    Rectangulo(f64, f64), // ancho, alto
    Triangulo { base: f64, altura: f64 }, // variante con campos
    Punto, // variante sin datos
}

impl Forma {
    fn area(&self) -> f64 {
        match self {
            Forma::Circulo(radio) => std::f64::consts::PI * radio *
radio,
            Forma::Rectangulo(ancho, alto) => ancho * alto,
            Forma::Triangulo { base, altura } => 0.5 * base *
altura,
            Forma::Punto => 0.0,
        }
    }
}

fn main() {
    let formas = vec![
        Forma::Circulo(3.0),
        Forma::Rectangulo(4.0, 5.0),
        Forma::Triangulo { base: 6.0, altura: 2.0 },
        Forma::Punto,
    ];

    for forma in &formas {
        println!("Área: {:.2}", forma.area());
    }
}
```

Diagrama mental: cómo se almacena un enum

Forma en memoria (stack):

discriminante (tag: u8)	datos de la variante activa (espacio = tamaño de la variante MÁS GRANDE)
----------------------------	--

Ejemplo: `Forma::Rectangulo(4.0, 5.0)`

tag	4.0	5.0
1	(f64)	(f64)

Rust reserva memoria suficiente para la variante más grande, más un discriminante (tag) que indica cuál variante está activa. Esto es lo que hace posible el match exhaustivo.

3.3 Pattern Matching con match

match es **exhaustivo**: el compilador te obliga a cubrir todos los casos posibles.

```
fn describir_numero(n: i32) -> String {
    match n {
        0 => String::from("cero"),
        1 | 2 | 3 => String::from("pequeño"), // múltiples patrones
    con |
        4..=9 => String::from("mediano"),      // rangos
        n if n < 0 => String::from("negativo"), // match guard
    (condición extra)
        _ => String::from("grande"),          // catch-all
    obligatorio
    }
}

fn main() {
    for n in [-5, 0, 2, 7, 100] {
        println!("{}", n, describir_numero(n));
    }
}
```

⚠ Error típico del compilador #2: match no exhaustivo

```
fn describir(n: i32) -> String {
    match n {
        0 => String::from("cero"),
        1 => String::from("uno"),
    } // ❌ falta cubrir el resto de i32
}
```

```
error[E0004]: non-exhaustive patterns: `i32::MIN..=-1_i32` and
`2_i32..=i32::MAX` not covered
--> src/main.rs:2:11
  |
2 |     match n {
  |           ^ patterns `i32::MIN..=-1_i32` and `2_i32..=i32::MAX`
not covered
  |
help: ensure that all possible cases are being handled by adding a
match arm
      with a wildcard pattern or an explicit pattern
```

Esto es una feature, no un bug del desarrollador. En otros lenguajes, un switch sin default simplemente no hace nada si no matchea (bug silencioso). En Rust, es **imposible olvidar un caso**: el compilador literalmente no te deja.

3.4 Option<T>: Adiós a null

Rust **no tiene null**. En su lugar, cualquier valor que "puede no existir" se modela con el enum de la librería estándar:

```
enum Option<T> {
    Some(T),
    None,
}

fn dividir(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 {
        None
    } else {
        Some(a / b)
    }
}
```

```
fn main() {
    let resultado = dividir(10.0, 2.0);

    // Opción 1: match exhaustivo
    match resultado {
        Some(valor) => println!("Resultado: {}", valor),
        None => println!("No se puede dividir por cero"),
    }

    // Opción 2: if let (cuando solo te importa un caso)
    if let Some(valor) = dividir(20.0, 4.0) {
        println!("Resultado: {}", valor);
    } else {
        println!("Error de división");
    }

    // Opción 3: métodos combinadores (muy idiomático)
    let resultado2 = dividir(15.0, 3.0).unwrap_or(0.0); // valor por
defecto si es None
    println!("{}", resultado2);

    let resultado3 = dividir(8.0, 0.0).map(|v| v * 2.0); //
transforma si Some, propaga None
    println!("{:?}", resultado3); // None
}
```

⚠ Error típico del compilador #3: Olvidar que Option<T> no es T

```
fn main() {
    let x: Option<i32> = Some(5);
    let y = x + 1; // ❌
}
```

```
error[E0369]: cannot add `{integer}` to `Option<i32>`
--> src/main.rs:3:15
```

```
3 |         let y = x + 1;
  |                   ^ - {integer}
  |                   |
  |                   Option<i32>
```

```
help: consider using `Option::expect` to unwrap the `i32` value,
panicking if the option is `None`
```

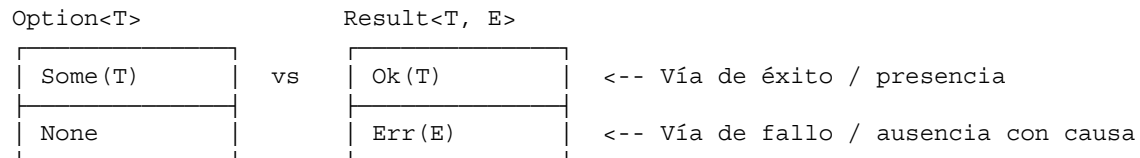
La lección central: no puedes "olvidarte" de manejar el caso None, como sí puedes olvidar chequear null en Java/JS y obtener un NullPointerException en producción a las 3am. El compilador te obliga a **desempaquetar** el Option explícitamente en algún punto — con match, if let, unwrap_or, ?, etc. El error de null-pointer, la causa #1 histórica de bugs en software, **es una clase de error que no existe en Rust seguro.**

3.4.b Dualidad Option<T> y Result<T, E>: Anatomía y Preview del Operador ?

Ambos tipos son enums fundamentales de la biblioteca estándar para modelar incertidumbre sin recurrir a excepciones o valores nulos:

- **Option<T>**: Representa la **posible ausencia** de un valor (Some(T) o None).
- **Result<T, E>**: Representa la **posibilidad de fallo** en una operación (Ok(T) o Err(E)).

Diagrama Mental: Comparativa de Estructura



El Operador ? en Option y Result

El operador ? se utiliza para la propagación temprana: si el valor es Some(v) o Ok(v), extrae v; si es None o Err(e), corta la ejecución y retorna inmediatamente ese estado hacia la función llamadora.

```
#[derive(Debug)]
struct Usuario {
    nombre: String,
    telefono: Option<String>,
}

// 1. Propagación temprana con Option<?>
fn obtener_codigo_area(usuario: &Usuario) -> Option<&str> {
    let tel = usuario.telefono.as_deref()?; // Si es None, retorna
    None inmediatamente
    if tel.starts_with('+') && tel.len() >= 4 {
        Some(&tel[0..4])
    } else {
        None
    }
}

// 2. Propagación temprana con Result<?, E>
fn parsear_puerto(input: &str) -> Result<u16, String> {
    let puerto: u16 = input.trim().parse().map_err(|_| "Formato
numérico inválido".to_string())?;
    if puerto > 1024 {
        Ok(puerto)
    } else {
        Err(String::from("El puerto debe ser superior a 1024"))
    }
}
```

```

fn main() {
    let user_sin_tel = Usuario { nombre: "Carlos".into(), telefono: None };
    let user_con_tel = Usuario { nombre: "Lucía".into(), telefono:
Some("+54911234567".into()) };

    println!("Área Carlos: {:?}", obtener_codigo_area(&user_sin_tel)); //
None
    println!("Área Lucía: {:?}", obtener_codigo_area(&user_con_tel)); //
Some("+549")

    println!("Puerto 8080: {:?}", parsear_puerto("8080"));
    println!("Puerto 80: {:?}", parsear_puerto("80"));
    println!("Puerto NaN: {:?}", parsear_puerto("abc"));
}

```

3.5 if let y while let

```

fn main() {
    let config_max: Option<u8> = Some(3);

    // if let: azúcar sintáctico para "si matchea esto, ejecuta, sino
ignora"
    if let Some(max) = config_max {
        println!("Máximo configurado: {}", max);
    }

    // while let: sigue iterando mientras el patrón matchee
    let mut pila = vec![1, 2, 3];
    while let Some(top) = pila.pop() {
        println!("Desapilado: {}", top);
    }
    println!("Pila vacía: {:?}", pila);
}

```

3.6 Pattern Matching Avanzado: Destructuring

```

struct Punto { x: i32, y: i32 }
fn main() {
    let p = Punto { x: 0, y: 7 };
    match p {
        Punto { x: 0, y: 0 } => println!("Origen"),
        Punto { x: 0, y } => println!("Eje Y en {}", y),
        Punto { x, y: 0 } => println!("Eje X en {}", x),
        Punto { x, y } => println!("En ({} , {})", x, y),
    }
    // Destructuring de tuplas anidadas
    let ((a, b), c) = ((1, 2), 3);
    println!("{}", a, b, c);
    // @ bindings: capturar el valor Y verificar un rango
    let n = 5;
    match n {
        valor @ 1..=5 => println!("Está en rango: {}", valor),
        _ => println!("Fuera de rango"),
    }
}

```

3.6.b Pattern Matching Avanzado: Omisión de Campos (..), @ Bindings y Match Guards

El sistema de coincidencia de patrones en Rust permite deconstruir datos de forma selectiva aplicando condiciones complejas en una única expresión.

1. Omisión de campos con ..

Permite extraer únicamente los campos relevantes de un struct o tupla grande, ignorando el resto.

2. Match Guards (if condición)

Añade una condición booleana adicional al brazo del match. El brazo solo se ejecutará si el patrón coincide **y** la guardia evalúa como true.

3. @ Bindings

Permite validar que un valor coincida con un patrón/rango y, al mismo tiempo, asignar ese valor a una variable local para usarlo en el cuerpo del brazo.

```
struct EventoRed {
    id: u64,
    ip_origen: String,
    puerto: u16,
    payload_size: usize,
    es_admin: bool,
}

fn auditar_evento(evento: EventoRed) {

    match evento {
        // Omisión de campos con `..` + Match Guard
        EventoRed { es_admin: true, .. } => {
            println!("Acceso de administrador verificado.");
        }

        // @ Binding combinado con Rango y Match Guard
        EventoRed { puerto: p @ 1..=1024, payload_size, .. } if
payload_size > 1000 => {
            println!("ALERTA: Payload anómalo ({payload_size} bytes) en
puerto privilegiado {}", p);
        }

        // @ Binding puro sobre el struct completo
        evento_regular @ EventoRed { .. } => {
            println!("Evento estándar #{} registrado desde {}",
evento_regular.id, evento_regular.ip_origen);
        }
    }
}
```

```
fn main() {  
    let evento1 = EventoRed {  
        id: 1,  
        ip_origen: "10.0.0.1".into(),  
        puerto: 80,  
        payload_size: 4096,  
        es_admin: false,  
    };  
    let evento2 = EventoRed {  
        id: 2,  
        ip_origen: "192.168.1.50".into(),  
        puerto: 9000,  
        payload_size: 128,  
        es_admin: false,  
    };  
    auditar_evento(evento1);  
    auditar_evento(evento2);  
}
```

Ejercicio Integrador — Módulo 3

Proyecto: Máquina de Estados de un Semáforo Inteligente + Figuras Geométricas

Crea cargo new modulo3_proyecto e implementa:

Parte A — Enum como máquina de estados:

```
enum Semaforo {
    Rojo,
    Amarillo,
    Verde,
}

impl Semaforo {
    fn siguiente(&self) -> Semaforo {
        // Implementa la transición: Rojo -> Verde -> Amarillo
        -> Rojo
        todo!()
    }
    fn duracion_segundos(&self) -> u32 {
        // Rojo: 30, Amarillo: 5, Verde: 25
        todo!()
    }
}
```

Simula 6 transiciones consecutivas con un loop, imprimiendo el estado y su duración en cada iteración.

Parte B — Sistema de figuras con Option:

1. Define un enum Figura con variantes Circulo(f64), Cuadrado(f64), Rectangulo(f64, f64).
2. Implementa fn perimetro(&self) -> f64 con match exhaustivo.
3. Implementa fn figura_mas_grande(figuras: &[Figura]) -> Option<&Figura> que devuelva None si el slice está vacío, o la referencia a la figura con mayor área si no lo está (necesitarás también un método area(&self) -> f64).
4. En main, usa if let para manejar el Option devuelto por figura_mas_grande sobre un vector vacío y sobre uno con elementos, mostrando ambos casos.

Parte C — Struct con datos personales:

Define un struct Empleado con nombre: String, salario: f64, departamento: Option<String> (el departamento es opcional porque hay empleados sin asignar aún). Escribe una función que imprima el departamento o "Sin asignar" usando unwrap_or_else o match.

Ejercicio Integrador 2

Proyecto: Router de Eventos Transaccionales Tipado

Crea cargo new router_eventos e implementa:

1. **Modelado con Newtypes:**
 - Define struct TransaccionId(u64) y struct ClientId(u64).
2. **Estructuras con Default:**
 - Define struct MetadataTransaccion con campos prioridad: u8, timestamp: u64 y reintentos: u8.
 - Implementa/Deriva Default donde prioridad = 1, timestamp = 0, y reintentos = 3.
3. **Enum de Dominio con Datos Asociados:**
 - Define enum Transaccion con variantes:
 - Pago { id: TransaccionId, cliente: ClientId, monto: f64, meta: MetadataTransaccion }
 - Reembolso { id: TransaccionId, motivo: String, monto: f64 }
 - BloqueoCuenta(ClientId)
4. **Función de Procesamiento con Pattern Matching Avanzado:**
 - Escribe fn procesar_transaccion(tx: Transaccion) -> Result<String, String> que maneje:
 - Pagos con montos ≤ 0.0 $\rightarrow$ Retorna Err("Monto inválido").
 - Pagos de alto valor (monto @ 10_000.0.. $f64::MAX$) $\rightarrow$ Retorna Ok indicando auditoría obligatoria.
 - Reembolsos ignorando la metadata $\rightarrow$ Retorna Ok con el id y motivo.
 - Bloqueo de cuenta $\rightarrow$ Retorna Ok con el ID del cliente.
5. **En main:**
 - Crea instancias de prueba utilizando ..Default::default() para la metadata, procesa cada evento y muestra cómo el operador ? permite verificar transacciones encadenadas de forma concisa.